

Zadanie 4. Rozszerzenia lokalnego przeszukiwania

Inteligentne Metody Optymalizacji

Maciej Szymaniak, 155830

Piotr Szymaczek, 155889

10 maja 2026

1 Opis zadania

Rozważamy zmodyfikowany problem komiwojażera. Dany jest zbiór n wierzchołków z symetryczną macierzą odległości d_{ij} oraz zyskiem p_i za odwiedzenie wierzchołka i . Należy wybrać podzbiór wierzchołków i ułożyć zamkniętą trasę przechodzącą przez wybrane wierzchołki, maksymalizując funkcję celu:

$$f(\mathcal{C}) = \sum_{i \in \mathcal{C}} p_i - \sum_{(i,j) \in \mathcal{C}} d_{ij}$$

Jako lokalne przeszukiwanie zastosowano **SLS** (Steepest Local Search) – najlepszą metodę z Zadania 3, realizującą pełne przeszukiwanie sąsiedztwa (ruchy ADD, REMOVE, 2-OPT). Praca zawiera implementację i porównanie następujących metaheurystyk:

- **MSLS** (Multiple Start Local Search) – 200 niezależnych startów z losowych rozwiązań, każdy zakończony SLS; zwraca globalne najlepsze rozwiązanie.
- **ILS** (Iterated Local Search) – jeden start, iteracyjna perturbacja bieżącego rozwiązania i ponowne SLS; limit czasu = średni czas MSLS.
- **LNS** (Large Neighborhood Search) – niszczenie ok. 30% trasy i naprawa heurystyką żalu 2-Regret, z opcjonalnym SLS; limit czasu = średni czas MSLS.
- **LNSa** – wariant LNS *bez* SLS po fazie naprawy.

Kod źródłowy dostępny pod adresem:

<https://github.com/masz00/Intelligent-Optimization-Methods/LocalSearchExtensions>

Rozwiązania zweryfikowane za pomocą SolutionCheckera: *TAK* (160/160 plików poprawnych).

2 Pseudokody algorytmów

Algorithm 1: SLS – Steepest Local Search (najlepsza metoda z Zadania 3)

Wejście: Trasa x , macierz odległości D , lista zysków p

Wyjście: Lokalnie optymalna trasa x

$improved \leftarrow$ **prawda**;

dopóki $improved$ **wykonaj**

$improved \leftarrow$ **fałsz**; $\Delta_{best} \leftarrow 0$; $m_{best} \leftarrow \emptyset$;

$n \leftarrow |x|$; $wTrasie[v] \leftarrow$ **fałsz** dla wszystkich v ; $wTrasie[x[i]] \leftarrow$ **prawda** dla

$i = 0, \dots, n-1$;

dla każdego $i = 0, \dots, n-1$ **wykonaj**

$v \leftarrow x[i]$; $w \leftarrow x[(i+1) \bmod n]$;

dla każdego wierzchołek u z $wTrasie[u] =$ **fałsz** **wykonaj**

 // Ruch ADD: wstaw u między v a w

$\Delta \leftarrow p_u - (d_{v,u} + d_{u,w} - d_{v,w})$;

jeżeli $\Delta > \Delta_{best}$ **to** $\Delta_{best} \leftarrow \Delta$;;

$m_{best} \leftarrow \text{Add}(u, v, w)$;

koniec

jeżeli $n > 3$ **to**

 // Ruch REMOVE: usuń v z trasy

$v_{prev} \leftarrow x[(i-1+n) \bmod n]$;

$\Delta \leftarrow -p_v + (d_{v_{prev},v} + d_{v,w} - d_{v_{prev},w})$;

jeżeli $\Delta > \Delta_{best}$ **to** $\Delta_{best} \leftarrow \Delta$;;

$m_{best} \leftarrow \text{Remove}(v)$;

koniec

koniec

dla każdego $i = 0, \dots, n-2$ **wykonaj**

dla każdego $j = i+2, \dots, n-1$, *pomiijając* ($i=0, j=n-1$) **wykonaj**

 // Ruch 2-OPT: zamień krawędzie (x_i, x_{i+1}) i (x_j, x_{j+1})

$\Delta \leftarrow (d_{x_i, x_{i+1}} + d_{x_j, x_{j+1}}) - (d_{x_i, x_j} + d_{x_{i+1}, x_{j+1}})$;

jeżeli $\Delta > \Delta_{best}$ **to** $\Delta_{best} \leftarrow \Delta$;;

$m_{best} \leftarrow \text{2-opt}(i, j)$;

koniec

koniec

jeżeli $\Delta_{best} > 0$ **to**

 Wykonaj ruch m_{best} na trasie x ;

$improved \leftarrow$ **prawda**;

koniec

koniec

zwróć x

Algorithm 2: MSLS – Multiple Start Local Search

Wejście: Macierz odległości D , lista zysków p

Wyjście: Najlepsze znalezione rozwiązanie x_{best}

```
 $x_{best} \leftarrow \emptyset; \quad f_{best} \leftarrow -\infty;$   
for  $i \leftarrow 1$  to 200 do  
     $x \leftarrow$  losowe rozwiązanie startowe;  
     $x \leftarrow \text{SLS}(x, D, p);$   
    jeżeli  $f(x) > f_{best}$  to  
         $f_{best} \leftarrow f(x); \quad x_{best} \leftarrow x;$   
    koniec  
end  
zwróć  $x_{best}$ 
```

Algorithm 3: Perturbacja dla ILS

Wejście: Trasa x , lista wszystkich wierzchołków V

Wyjście: Zaburzona trasa x

```
 $n \leftarrow |x|;$   
jeżeli  $n \geq 4$  to  
    for  $i \leftarrow 1$  to 3 do  
        // 3 losowe ruchy 2-OPT  
         $i_1 \leftarrow$  losowy indeks z  $[0, n); \quad i_2 \leftarrow$  losowy indeks z  $[0, n);$   
        jeżeli  $|i_1 - i_2| > 1$  to odwróć segment  $x[\min(i_1, i_2) \dots \max(i_1, i_2)];$   
    end  
koniec  
 $r \leftarrow \min(2, \max(0, |x| - 3));$   
for  $i \leftarrow 1$  to  $r$  do  
    // Usuń co najwyżej 2 losowe wierzchołki, zachowując min. 3 węzły  
    Usuń losowy element z  $x;$   
end  
for  $i \leftarrow 1$  to 2 do  
    // Wstaw 2 nowe losowe wierzchołki spoza trasy  
     $u \leftarrow$  losowy wierzchołek z  $V \setminus x;$   
    Dołącz  $u$  do końca  $x;$   
end  
zwróć  $x$ 
```

Algorithm 4: ILS – Iterated Local Search

Wejście: Macierz odległości D , lista zysków p , limit czasu T

Wyjście: Najlepsze znalezione rozwiązanie x_{best}

$x \leftarrow$ losowe rozwiązanie startowe;

$x \leftarrow \text{SLS}(x, D, p)$;

$x_{best} \leftarrow x$; $f_x \leftarrow f(x)$; $f_{best} \leftarrow f(x)$;

dopóki czas() $< T$ **wykonaj**

$y \leftarrow x$; // Kopia bieżącego rozwiązania

$y \leftarrow \text{Perturbacja}(y, V)$; // $3 \times 2\text{-OPT} + \text{usuń} \leq 2 + \text{wstaw } 2$

$y \leftarrow \text{SLS}(y, D, p)$;

$f_y \leftarrow f(y)$;

jeżeli $f_y > f_x$ **to**

$x \leftarrow y$; $f_x \leftarrow f_y$; // Kryterium akceptacji: tylko poprawa

koniec

jeżeli $f_y > f_{best}$ **to**

$x_{best} \leftarrow y$; $f_{best} \leftarrow f_y$; // Zapamiętaj globalny rekord

koniec

koniec

zwróć x_{best}

Algorithm 5: RepairLNS – naprawa metodą heurystyki żalu 2-Regret

Wejście: Uszkodzona trasa x , macierz odległości D , lista zysków p

Wyjście: Naprawiona trasa x o rozmiarze N_{all}

$\text{odwiedzone}[v] \leftarrow \text{fałsz}$ dla wszystkich v ; $\text{odwiedzone}[x[i]] \leftarrow \text{prawda}$ dla
 $i = 0, \dots, |x|-1$;

dopóki $|x| < N_{all}$ **wykonaj**

$u^* \leftarrow -1$; $r_{\max} \leftarrow -\infty$;

dla każdego wierzchołek u z $\text{odwiedzone}[u] = \text{fałsz}$ **wykonaj**

$c_{\min} \leftarrow +\infty$; $c_{\min 2} \leftarrow +\infty$; $pos^* \leftarrow -1$;

for $j \leftarrow 0$ **to** $|x|-1$ **do**

$a \leftarrow x[j]$; $b \leftarrow x[(j+1) \bmod |x|]$;

$c \leftarrow d_{a,u} + d_{u,b} - d_{a,b} - p_u$; // Koszt netto wstawienia u między a i b

jeżeli $c < c_{\min}$ **to**

$c_{\min 2} \leftarrow c_{\min}$; $c_{\min} \leftarrow c$; $pos^* \leftarrow j+1$;

koniec

w p.p. jeżeli $c < c_{\min 2}$ **to**

$c_{\min 2} \leftarrow c$;

koniec

end

jeżeli $c_{\min 2} - c_{\min} > r_{\max}$ **to**

$r_{\max} \leftarrow c_{\min 2} - c_{\min}$; $u^* \leftarrow u$; $wPos \leftarrow pos^*$;

koniec

koniec

jeżeli $u^* = -1$ **to przerwij**;

 Wstaw u^* do x na pozycję $wPos$; $\text{odwiedzone}[u^*] \leftarrow \text{prawda}$;

koniec

zwróć x

Algorithm 6: LNS – Large Neighborhood Search

Wejście: Macierz odległości D , lista zysków p , limit czasu T , flaga *useLS* (LNS: **prawda**; LNSa: **fałsz**)
Wyjście: Najlepsze znalezione rozwiązanie x

$x \leftarrow$ losowe rozwiązanie startowe;
 $x \leftarrow \text{SLS}(x, D, p)$;
 $f_{best} \leftarrow f(x)$;

dopóki $\text{czas}() < T$ **wykonaj**

$y \leftarrow x$;

// Faza Destroy: usuń ok. 30% węzłów w $s \in \{3, 4\}$ losowych segmentach
 $s \leftarrow 3 + \text{rand}(0, 1)$; $\ell \leftarrow \lfloor (|y| \cdot 3/10) / s \rfloor$;

for $i \leftarrow 1$ **to** s **do**

jeżeli $|y| \leq \ell + 3$ **to przerwij**;

$start \leftarrow$ losowy indeks z $[0, |y|)$;

for $j \leftarrow 1$ **to** ℓ **do**

Usun $y[start \bmod |y|]$;

end

end

// Faza Repair: uzupełnij trasę heurystyką żalu 2-Regret
 $y \leftarrow \text{RepairLNS}(y, D, p)$;

jeżeli *useLS* **to**

$y \leftarrow \text{SLS}(y, D, p)$; // Opcjonalne SLS (pomijane w LNSa)

koniec

jeżeli $f(y) > f_{best}$ **to**

$x \leftarrow y$; $f_{best} \leftarrow f(y)$; // Akceptuj wyłącznie poprawy

koniec

koniec
zwróć x

3 Wyniki eksperymentów

Eksperyment przeprowadzono, wykonując 20 niezależnych uruchomień każdego algorytmu. Limit czasu ILS i LNS wyznaczono dynamicznie jako średni czas 20 pełnych przebiegów MSLS (≈ 1418 ms dla TSPA, ≈ 1577 ms dla TSPB).

3.1 Instancja TSPA

Algorytm	Wynik Avg (Min – Max)	Avg Iteracji	Avg Czas [ms]
MSLS	7691 (7437 – 7997)	200	1418.16
ILS	8135 (7531 – 8420)	3807	1418.33
LNS	8134 (7593 – 8571)	314	1420.74
LNSa	5939 (3832 – 7213)	1022	1419.01

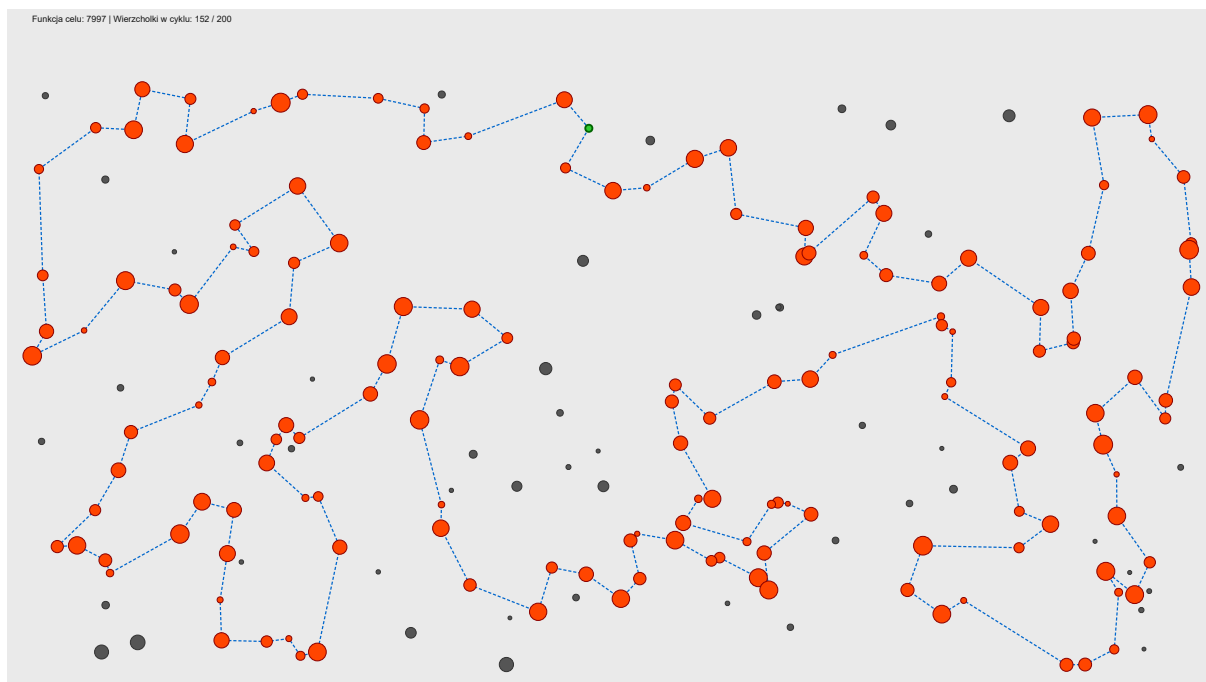
Tabela 1: Wyniki dla instancji TSPA (20 uruchomień)

3.2 Instancja TSPB

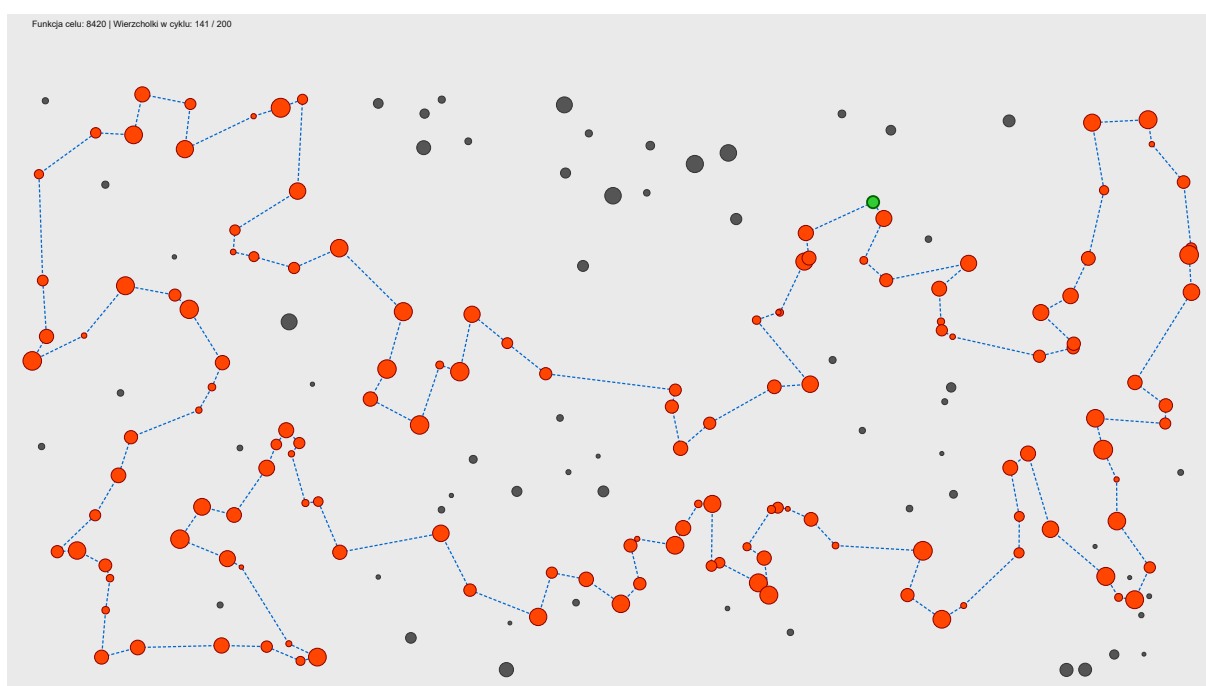
Algorytm	Wynik Avg (Min – Max)	Avg Iteracji	Avg Czas [ms]
MSLS	19369 (19003 – 19791)	200	1576.77
ILS	19786 (19430 – 20045)	3999	1577.02
LNS	19768 (19591 – 19967)	463	1578.58
LNSa	17770 (16371 – 19247)	1397	1577.43

Tabela 2: Wyniki dla instancji TSPB (20 uruchomień)

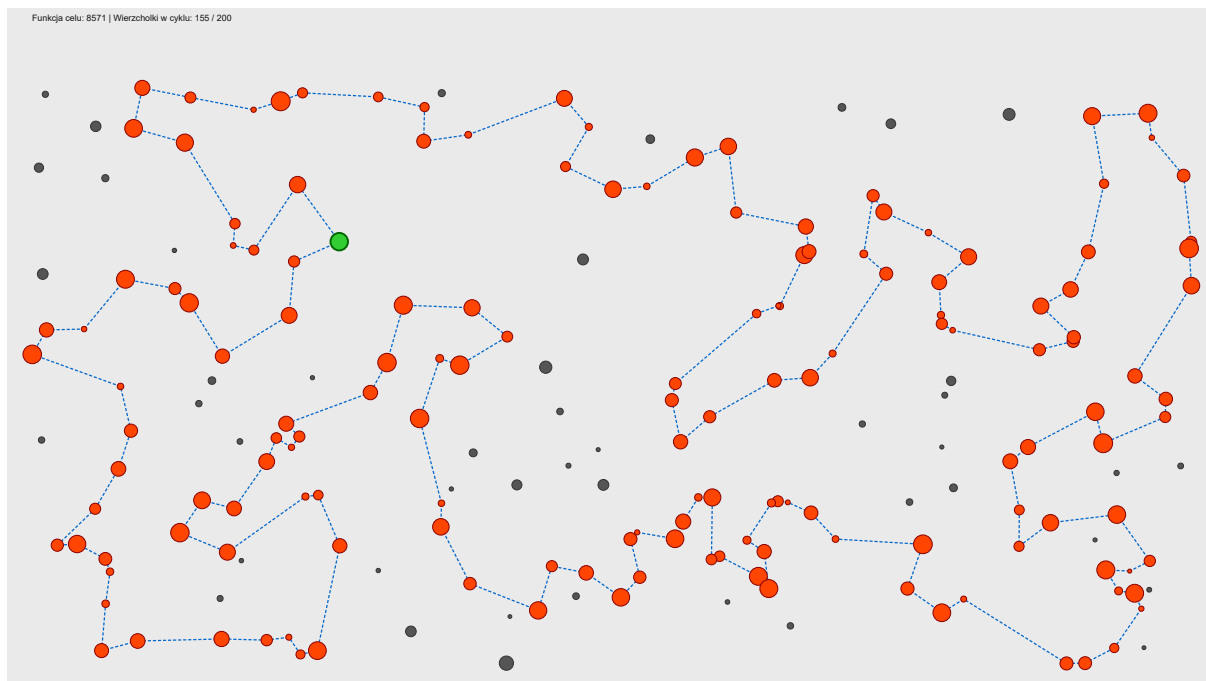
4 Wizualizacje najlepszych rozwiązań



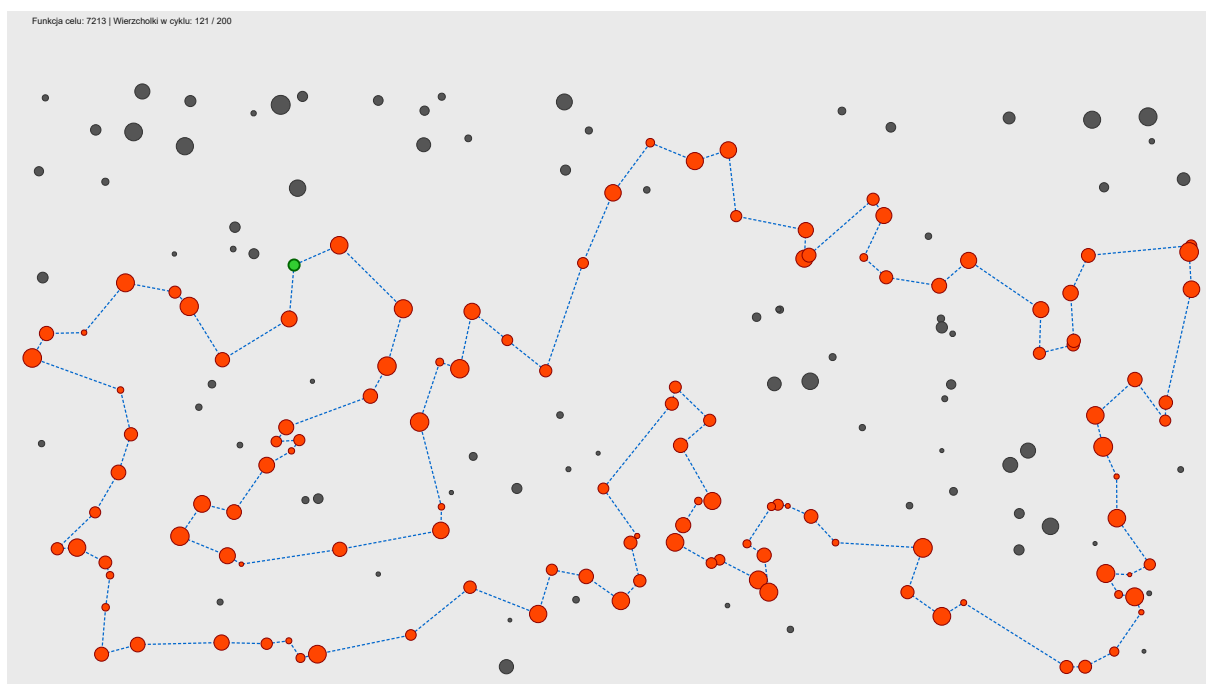
Rysunek 1: TSPA – Metoda MSLS (najlepszy wynik: 7997)



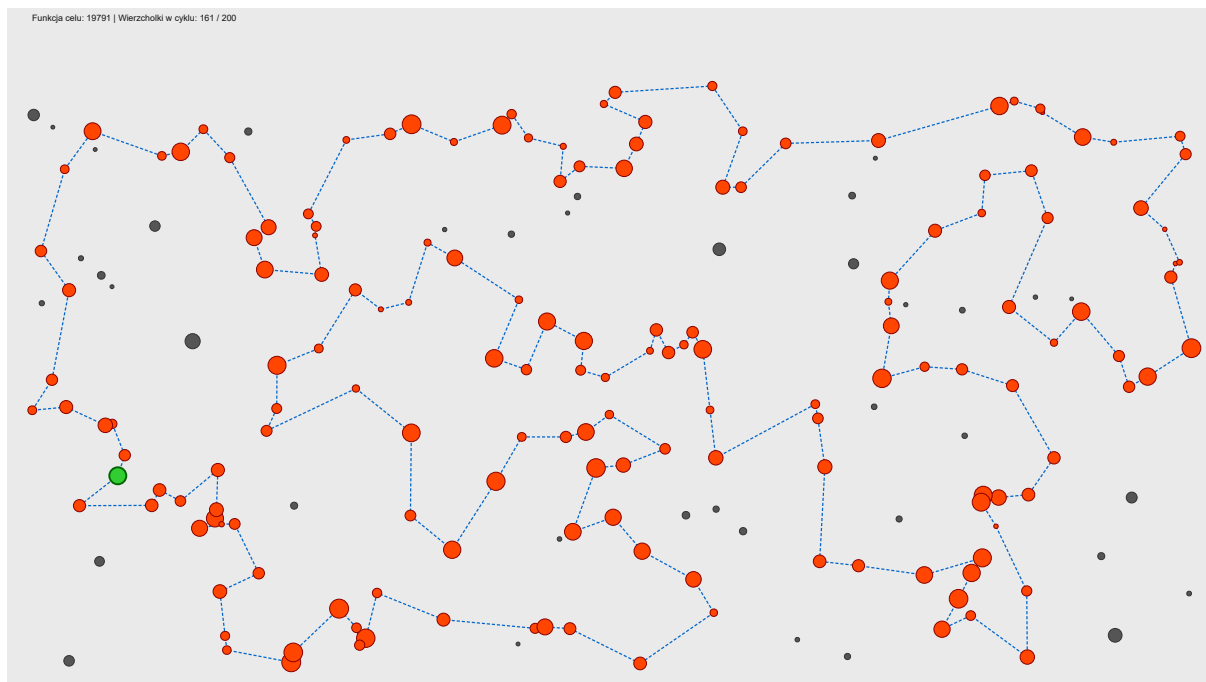
Rysunek 2: TSPA – Metoda ILS (najlepszy wynik: 8420)



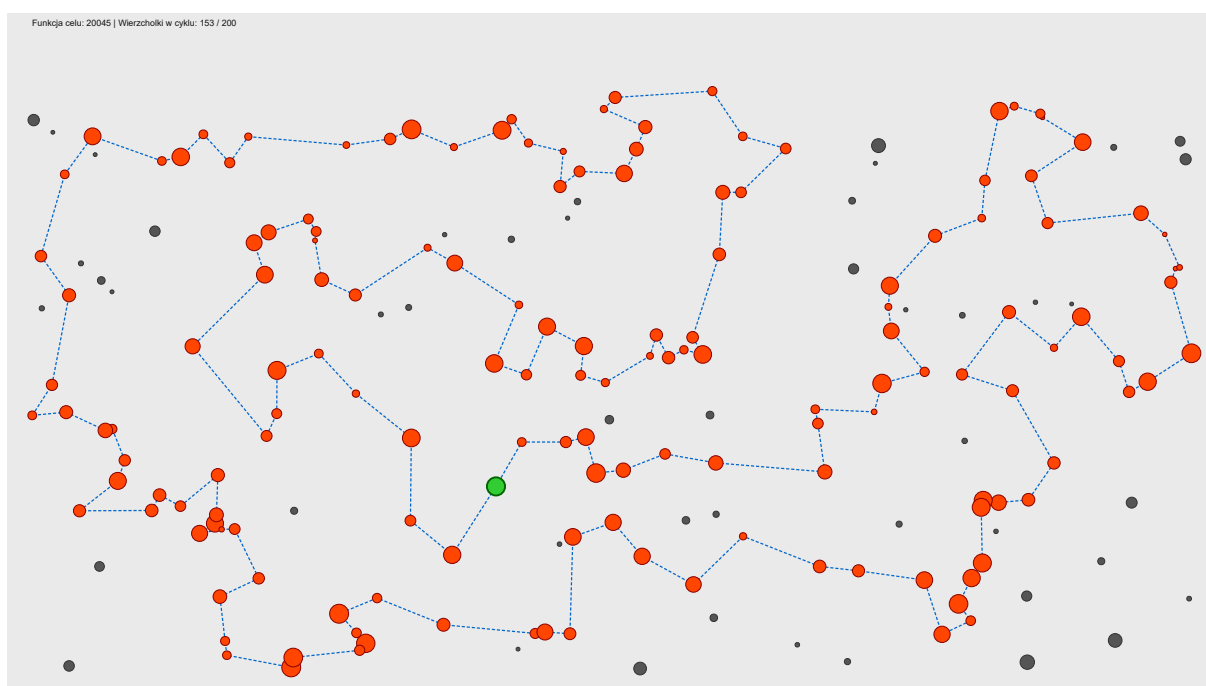
Rysunek 3: TSPA – Metoda LNS (najlepszy wynik: 8571)



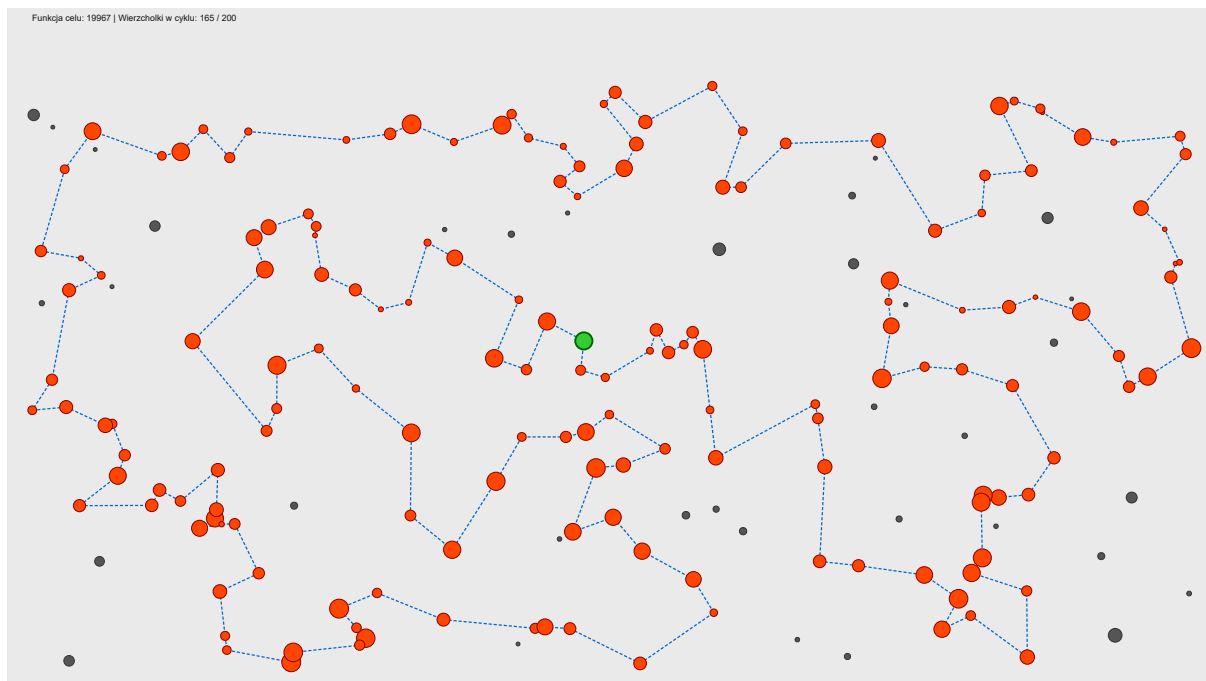
Rysunek 4: TSPA – Metoda LNSa (najlepszy wynik: 7213)



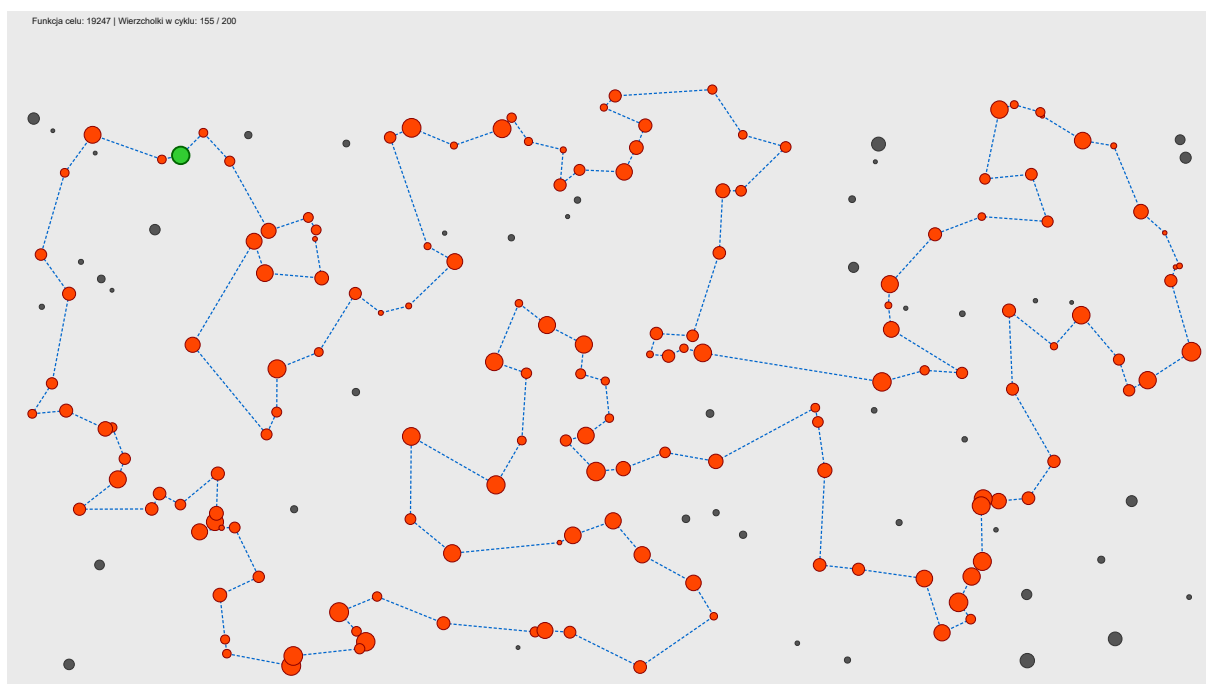
Rysunek 5: TSPB – Metoda MSLS (najlepszy wynik: 19791)



Rysunek 6: TSPB – Metoda ILS (najlepszy wynik: 20045)



Rysunek 7: TSPB – Metoda LNS (najlepszy wynik: 19967)



Rysunek 8: TSPB – Metoda LNSa (najlepszy wynik: 19247)

5 Wnioski

1. **ILS i LNS zdecydowanie lepsze od MSLS:** Obie metody iteracyjne uzyskały średnie wyniki wyższe o ok. 6% (TSPA) i ok. 2% (TSPB) w stosunku do MSLS przy tym samym budżecie czasowym. Potwierdza to, że przenoszenie wiedzy między iteracjami (przez perturbację lub Destroy-Repair) jest bardziej efektywne niż wielokrotny restart od zera.
2. **ILS i LNS praktycznie równorzędne:** Średnie wyniki obu metod są prawie identyczne: 8135 vs 8134 (TSPA) i 19786 vs 19768 (TSPB). ILS wykonuje ok. 3800–4000 iteracji w czasie limitu, podczas gdy LNS tylko 314–463. ILS korzysta z tego, że perturbacja jest lekka – SLS startuje z bliskiego lokalnego optimum i zbiega w ułamku czasu pełnego biegu. LNS z kolei generuje radykalnie różne rozwiązania dzięki Destroy-Repair, co widać w najlepszym absolutnym wyniku dla TSPA: LNS 8571 vs ILS 8420.
3. **MSLS – brak synergii między startami:** 200 niezależnych startów SLS daje węższy zakres wyników (mały rozstęp min–max) lecz niższą średnią, bo każda iteracja kosztuje tyle samo co pojedyncze wywołanie SLS w ILS/LNS, a wiedza o poprzednich lokalnych optimach jest porzucana.
4. **Krytyczna rola SLS po fazie Repair w LNS:** LNSa (bez SLS) wypada drastycznie gorzej – 5939 (TSPA) i 17770 (TSPB). Faza Repair (2-Regret) wstawia wszystkie $N = 200$ węzłów do trasy, w tym te nierentowne, których usunięcie wymaga osobnego kroku SLS. Bez tego kroku każda iteracja LNSa degradowe rozwiązanie i aktualizacje stanu x są rzadkie; stąd paradoksalnie wysoka liczba iteracji (1022/1397) przy niskich wynikach.